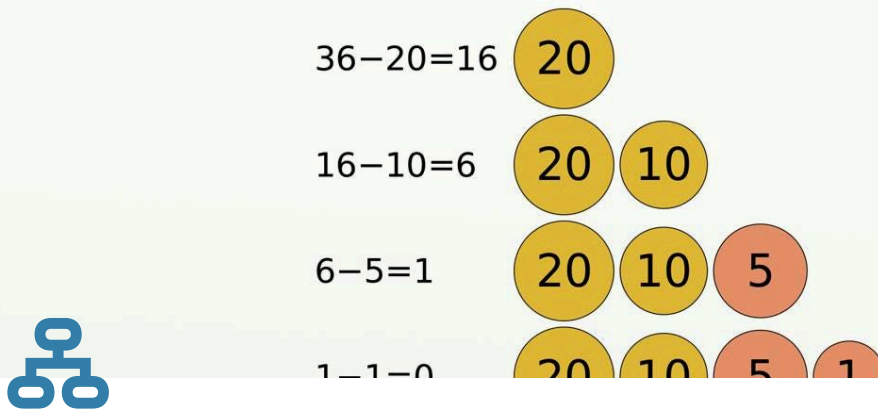


Greedy algorithm



1. Greedy Approach

"Take what seems best now and hope for the best overall"

Previous Section:

 [0. Data Structures & Algorithms](#)

Contents:

[1. What is a Greedy Approach?](#)

[2. Define a Greedy Approach Algorithm](#)

[3. Greedy Approach Example: Money Exchange Problem](#)

[3. Applications of Greedy Approach](#)

[4. Advantages of Greedy Approach](#)

[5. Disadvantages of Greedy Approach](#)

[Summary](#)

[References](#)

The name itself already sounds suspiciously human. **Greedy.**

When people hear the word "greedy", they often imagine someone who wants results immediately. Someone impatient. Someone ambitious enough to grab whatever looks best at the moment without stopping to think too far ahead. In real life, humans do this all the time. Sometimes it works brilliantly. Sometimes it becomes a disaster later.

Interestingly enough, Greedy Algorithms operate with almost the exact same mindset.

A greedy algorithm always tries to take the *currently best-looking option*. It does not pause to simulate every future possibility. It does not carefully explore every path like some overthinking philosopher questioning reality itself. Instead, it simply says:

■ ***"This choice looks the best right now, so I'm taking it."***

And then it keeps moving forward. This makes greedy algorithms extremely fast, simple, and often surprisingly effective. But there is also a catch hidden beneath that simplicity, just because something looks optimal *locally* does not always mean it will become optimal *globally*. Humans are kinda like this too, honestly.

1. What is a Greedy Approach?

Greedy Algorithm. Greedy Approach. Same thing. Don't overcomplicate the terminology.

A Greedy Algorithm is a problem-solving technique that makes the best possible choice at each step in hopes of eventually reaching the best overall solution. Instead of exploring every possible combination, the algorithm commits to decisions immediately and never revisits them later.

In simpler terms:

- It chooses what appears to be the best option *right now*.
- It assumes this choice will somehow contribute to the best final outcome.
- It never backtracks or changes previous decisions.
- It focuses on immediate gain rather than long-term planning.

This is why greedy algorithms are often described as operating with a **local optimization strategy**.

- The algorithm does not ask: *"What if this becomes bad later?"*
- It simply asks: *"What is the best thing I can do at this exact moment?"*

And honestly, that makes greedy algorithms feel strangely human.

The Philosophy Behind Greedy Algorithms - Greedy algorithms follow a very direct ideology:

┃ *"Take the best-looking option available now and continue forward."*

No hesitation. No reconsideration. No existential crisis halfway through the execution.

This is what makes greedy approaches extremely efficient for certain problems. Since the algorithm does not waste time exploring every possibility, it can often produce solutions very quickly.

However, there is an important limitation. A greedy algorithm does **not** guarantee the perfect solution for every problem. Why? Because sometimes the "best choice right now" eventually leads to a terrible outcome later.

Imagine climbing a mountain in heavy fog. A greedy strategy would always walk toward the steepest upward path visible nearby. Most of the time, that sounds reasonable. But occasionally, that path might lead to a dead end or a cliff while another slightly less optimal-looking path would have eventually reached the true peak.

→ That is the weakness of greedy thinking. It is powerful. It is fast. But it can also be shortsighted.

When Can Greedy Algorithms Actually Work?

Not every problem can be solved correctly using a greedy approach. For greedy algorithms to work properly, the problem usually needs two important properties.

- **Greedy Choice Property:** This means an optimal solution can be achieved by repeatedly making the best immediate choice without needing to reconsider previous decisions later.
 - In other words: If taking the best option now eventually leads to the best overall answer, the greedy approach works.

- Once the choice is made, the algorithm commits to it permanently. No regrets. No undo button.
- **Optimal Substructure:** This means the overall optimal solution can be built from optimal solutions to smaller subproblems.
 - Basically, if solving small pieces correctly helps construct the correct larger solution, the greedy method becomes possible. Many famous greedy problems rely heavily on this property.

A Human Example of Greedy Thinking: Imagine you are trying to buy dinner late at night. You have three choices: McDonald's, Burger King, Popeyes. Now normally, this sounds easy.

But suddenly there are conditions (or constraints):

- Your gas tank is almost empty.
- There are no nearby gas stations.
- Traffic conditions are different for each route.
- You are hungry enough to start questioning your life decisions.

Now your brain immediately begins evaluating the *best immediate option*.

- Maybe Popeyes is your favorite, but it is too far away.
- Maybe Burger King has shorter traffic.
- Maybe McDonald's is closest and guarantees you can still drive home safely.

At this point, you are unconsciously using greedy reasoning. You are not calculating every future possibility mathematically. You are simply choosing the option that seems best *right now* under your constraints.

And honestly, this is why many people say greedy algorithms resemble natural human decision-making. Humans often solve problems through immediate optimization.

- Finding the fastest route.
- Organizing objects into a bag.
- Choosing the nearest available resource.
- Trying to maximize reward while minimizing effort.

⇒ Greedy algorithms imitate this behavior computationally.

The Important Catch About "Optimal" - Now here comes the most important philosophical trap. People hear the word **optimal** and automatically assume: *"Oh, so it means correct?"*

→ **Not necessarily.** Greedy algorithms do not guarantee absolute perfection.

They attempt to find the *best probable solution* using immediate decisions. Sometimes that solution truly becomes globally optimal. Other times, it only becomes "good enough".

And this is a very important distinction. Greedy algorithms do not see the world in strict terms like: True or False; Correct or Incorrect; Perfect or Failure. Instead, they operate more like:

┆ *"This currently appears to be the best move available."*

That is the essence of greed. And strangely enough, that mindset powers some of the most efficient algorithms in Computer Science and Machine Learning.

2. Define a Greedy Approach Algorithm

Now that we understand the philosophy behind greedy algorithms, the next question becomes:

┆ *"How do we actually build one?"*

Surprisingly, greedy algorithms are usually very simple in structure. They do not require massive recursive trees, painful backtracking, or exploring every possible future outcome. Instead, greedy algorithms move forward step by step, constantly choosing what appears to be the best immediate option.

The general process of designing a greedy algorithm can usually be broken into several stages.

- **Step 1: Clearly Define the Problem**

Before doing anything, the problem itself must be understood clearly:

What are the inputs? What is the desired output? What exactly are we trying to optimize?

Optimization is extremely important in greedy problems because greedy algorithms are all about trying to maximize or minimize something.

For example: Minimize cost; Maximize profit; Find shortest path; Use minimum resources; Save maximum time.

Without a clear objective, the algorithm has no idea what "best choice" even means.

- **Step 2: Determine the Best Local Choice**

This is the heart of the greedy approach. At every step, the algorithm asks:
"What is the best possible move I can make right now?"

The definition of "best" depends entirely on the problem. Sometimes "best" means: Largest value; Smallest weight; Shortest distance; Highest profit; Fastest route.

The algorithm does not attempt to analyze every future consequence. It simply chooses the currently most attractive option according to the optimization goal.

- **Step 3: Make the Choice and Update the State**

Once the greedy choice is selected, the algorithm commits to it immediately. No second thoughts. No regret. No emotional recovery phase. The system state is then updated accordingly.

For example: Remaining money decreases; Remaining capacity changes; Remaining distance becomes smaller; Remaining tasks are updated

This updated state becomes the starting point for the next greedy decision.

- **Step 4: Repeat Until the Problem is Solved**

The algorithm continues making greedy choices repeatedly until the objective is fully achieved.

Eventually: The destination is reached; The bag becomes full; The schedule is completed; The amount becomes zero.

At that point, the greedy algorithm terminates with its final solution.

3. Greedy Approach Example: Money Exchange Problem

Let us look at one of the most classic greedy examples: returning money using the minimum number of bills.

Suppose the available banknote denominations are:

$$[500, 100, 50, 20, 10, 5, 2, 1]$$

Now imagine somebody gives you 1000, and you need to return 657. The goal is simple: Use the minimum possible number of banknotes. Now watch how greedy thinking works.

- **Step 1: Choose the Largest Possible Denomination**

The greedy strategy immediately takes the biggest bill available that does not exceed the remaining amount.

The largest usable denomination is 500 $\rightarrow 657 // 500 = 1$

We take one 500 note. Remaining amount: $657 - 500 = 157$

Now the algorithm moves forward. But notice something important: It never questions whether using the 500 note was correct. It simply commits to it. That is pure greedy behavior.

- **Step 2: Repeat the Process**

Now the remaining amount is 157. The largest usable denomination becomes 100.

$$\Rightarrow 157 // 100 = 1$$

Take one 100 note. Remaining amount: $157 - 100 = 57$. Continue again.

- **Step 3: Continue Greedily**

Now: $57 // 50 = 1$. Remaining = 7

Then: 20 is too large $\rightarrow$ skip; 10 is too large $\rightarrow$ skip

Next: $7 // 5 = 1$. Remaining = 2

Finally: $2//2 = 1$. Remaining = 0 → The problem is solved.

- **Final Result** - The denominations used are: $[1, 1, 1, 0, 0, 1, 1]$

Which corresponds to:

$$(1 \times 500); (1 \times 100); (1 \times 50); (1 \times 20); (1 \times 10); (1 \times 5); (1 \times 2)$$

And somehow, through pure greed and confidence, the algorithm successfully minimized the number of bills. Honestly, this is why greedy algorithms are so satisfying to watch. They feel incredibly natural.

3. Applications of Greedy Approach

Greedy algorithms are used almost everywhere, both in computer science and in real-life decision making. Their simplicity and speed make them extremely useful, especially when finding the perfect solution would take too much time.

In many cases, greedy algorithms are not trying to achieve perfection. They are trying to achieve:

┆ *"A very good answer quickly."*

And surprisingly, that is often enough.

- **Finding Near-Optimal Solutions**

Some computational problems are extremely difficult to solve perfectly. Problems classified as NP-Hard, such as the Traveling Salesman Problem (TSP), can become computationally explosive as input sizes increase.

Greedy algorithms are commonly used to produce solutions that are *close* to optimal without requiring absurd amounts of computation time.

Instead of searching every possible route, the algorithm keeps selecting the currently most promising option. Fast. Practical. Good enough.

- **Network Design**

Greedy algorithms are heavily used in networking problems. Examples include: Minimum Spanning Trees; Shortest Path Problems; Maximum Flow Networks.

When designing systems such as internet routing, road systems, or communication infrastructure, greedy methods help efficiently determine low-cost and high-performance connections.

Algorithms like Dijkstra's Algorithm and Prim's Algorithm are famous greedy-based solutions.

- **Machine Learning** - Interestingly enough, greedy strategies also appear inside Machine Learning. For example:
 - **Feature Selection:** A greedy algorithm may repeatedly select the feature that improves prediction performance the most at the current stage.
 - **Clustering and Classification:** Greedy strategies can help optimize grouping decisions or class assignments step by step.

Machine learning systems often operate under limited resources and enormous datasets, so greedy optimization becomes extremely practical.

- **Financial Optimization**

Greedy algorithms are also used in financial systems. Examples include: Portfolio optimization; Investment selection; Risk management

A greedy strategy might continuously select assets that currently provide the best expected return under specific constraints.

Of course, financial markets are chaotic and unpredictable, which means greed can sometimes backfire horribly. Which is honestly very realistic.

- **Game Theory**

Games such as chess, poker, or strategic simulations frequently use greedy-like evaluations. At each move, the system attempts to identify:

┃ *"What is the strongest move I can make right now?"*

Although advanced AI systems usually combine many techniques together, greedy reasoning often forms an important foundation for move evaluation and tactical decision making.

- **Real-Life Problems**

Greedy algorithms are deeply connected to everyday optimization problems. Some famous examples include: Knapsack Problem; Huffman Coding; Minimum Spanning Trees; Shortest Path Finding; Resource Allocation; Scheduling Problems

In reality, humans constantly make greedy decisions without even realizing it. Finding the fastest route home. Choosing the shortest queue. Packing the most useful items first. We are all secretly running greedy heuristics in our heads.

4. Advantages of Greedy Approach

Greedy algorithms have many strengths that make them popular.

- **Simplicity:** Greedy algorithms are usually easy to understand and implement. Their logic is straightforward, which is choosing the best immediate option and continue. This simplicity makes them excellent teaching tools and practical solutions.
- **Fast Execution:** Because greedy algorithms avoid exploring every possibility, they are often significantly faster than exhaustive search methods. Less thinking. More action.
- **Efficient for Certain Problems:** When a problem satisfies the greedy choice property and optimal substructure, greedy algorithms can produce extremely efficient and even optimal solutions. Some famous problems are solved perfectly using greed. Which honestly feels almost unfair.
- **Useful Approximation Technique:** Sometimes finding the exact optimal solution would take too long. In those situations, greedy algorithms become powerful approximation tools capable of producing very good results quickly.
- **Broad Applicability:** Greedy strategies appear across many fields include Computer Science; Economics; Operations Research; Artificial Intelligence;

Logistics; Networking; Finance. Their flexibility makes them one of the most widely used algorithmic paradigms.

- **Real-Time Decision Making:** Greedy algorithms are excellent for systems that must react immediately. Examples include Scheduling systems; Resource allocation; Traffic routing; Dynamic networking. Since decisions are made instantly, they work well in real-time environments.
- **Good Starting Point:** Many advanced optimization techniques begin with a greedy solution before refining it further using more sophisticated methods. Greedy often acts as *"A strong first guess."*

5. Disadvantages of Greedy Approach

Despite their strengths, greedy algorithms also have major weaknesses. And honestly, these weaknesses can become catastrophic depending on the problem.

- **No Guaranteed Optimal Solution:** This is the biggest issue. Greedy algorithms can produce suboptimal answers because they only focus on local decisions instead of long-term consequences. Sometimes the algorithm becomes trapped in a "good-looking" path that ultimately leads to a worse overall result.
- **Extremely Dependent on Problem Structure:** Greedy algorithms only work properly when the problem itself supports greedy reasoning. If the local choice does not naturally lead toward the global optimum, the algorithm may fail badly.
- **Sensitive to Choice Criteria:** The entire algorithm depends on how "best choice" is defined. If the selection criteria are flawed, the final solution may become terrible. A small mistake in greedy logic can completely change the outcome.
- **Preprocessing May Be Required:** Some problems require significant transformation before a greedy strategy can even be applied. This preprocessing may reduce the simplicity advantage.
- **Not Suitable for Every Problem:** Certain problems depend heavily on future consequences, input order, or complex constraints. Greedy algorithms struggle badly in these situations. Examples include Bin Packing Problems; Complex Constraint Optimization; Multi-stage Planning Problems

- **Sensitive to Small Changes:** Greedy algorithms can sometimes behave unpredictably. Tiny changes in input may completely alter the sequence of greedy decisions, resulting in dramatically different outputs. This instability can become problematic in sensitive systems.

Summary

This section introduced the core idea behind the Greedy Approach and explored how greedy algorithms operate through immediate optimization.

A greedy algorithm repeatedly chooses the best-looking option available at the current step without reconsidering previous decisions. This makes greedy methods simple, fast, and highly practical for many optimization problems.

You also explored:

- The process of designing a greedy algorithm
- The classic money exchange example
- Real-world applications of greedy methods
- The strengths and weaknesses of greedy thinking

Most importantly, greedy algorithms teach an important lesson about optimization itself:

The “best immediate decision” is not always the perfect decision, but sometimes it is good enough to change everything.

References

<https://www.geeksforgeeks.org/dsa/introduction-to-greedy-algorithm-data-structures-and-algorithm-tutorials/>

<https://www.codecademy.com/article/greedy-algorithm-explained>

https://www.youtube.com/watch?v=ARvQcqJ_-NY

These next sections dive into some of the most iconic optimization problems in Computer Science. Where greed can be used to achieve a global optimum

by making locally optimal choices at each stage, it often provides the most efficient solution.

🔗 1.1. Knapsack Problem

🔗 1.2. Minimum Cost Spanning Tree

Next Section:

🔗 2. Divide and Conquer